

Spring Security & JWT Token Generation

Complete Flow Explained with a Real-World Login Example (TaskManager App)

Contents:

1. What Actually Happens When You "Login"
2. The Full Diagram, Explained Block by Block
3. Real-World Example Project Setup
4. Step-by-Step Code Walkthrough (mapped to the diagram)
5. Full Request/Response Trace
6. JWT Structure Refresher
7. Interview Q&A
8. Cheat Sheet

1. What Actually Happens When You "Login"

When a user submits a username and password, Spring Security doesn't just check a password against a database directly. It runs the request through a chain of specialised objects, each with one job. Once the identity is verified, we generate a **JWT (JSON Web Token)** that the client will use to prove who they are on every future request — without the server needing to remember a session.

Real-world analogy: think of logging into an office building. You show your ID card (username/password) to the **reception desk** (AuthController). Reception doesn't personally verify your identity — it calls **Security HQ** (AuthenticationManager), who forwards your ID to the specific **verification officer** (DaoAuthenticationProvider) trained for employee IDs. That officer calls **HR** (UserDetailsService) to fetch your real record from the **employee database**, then compares your password's hash using the office's approved **hashing machine** (PasswordEncoder). If it matches, HQ issues you a **visitor badge** (JWT) that you flash at every door (API endpoint) instead of showing your ID again and again.

2. The Full Diagram, Explained Block by Block

JWT Login Flow — AuthController to Token Generation

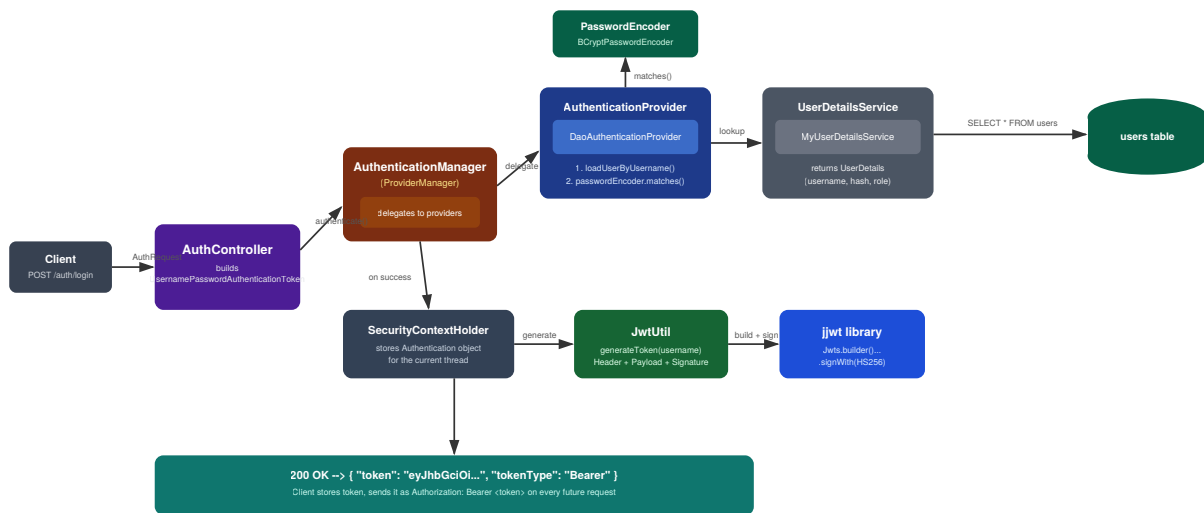


Diagram Block	What it really is	Its one job
AuthController	Your REST controller	Receives raw username/ password, wraps it into a token object, and returns the final JWT
AuthenticationManager	Interface, implemented by ProviderManager	Doesn't verify anything itself — finds the right provider and delegates to it
AuthenticationProvider	DaoAuthenticationProvider (one of many: OAuth2, LDAP, DAO)	Knows the actual algorithm to verify a username/password pair
UserDetailsService	Your own class, e.g. MyUserDetailsService	Fetches the user record (hashed password + roles) from the database
PasswordEncoder	BCryptPasswordEncoder	Hashes/compares passwords — raw password is never compared with ==
SecurityContextHolder	Thread-local storage	Holds the "who is currently authenticated" object for this request thread
JwtUtil	Your own utility class	Decides what claims go in the token and how long it lives

jjwt library	Third-party library (io.jsonwebtoken)	Actually builds and cryptographically signs the token string
--------------	---------------------------------------	--

3. Real-World Example Project Setup

We'll build the login API for a small **TaskManager** app. A user sends their username/password to POST /api/auth/login and gets back a JWT to use on all future requests.

3.1 Entity — how the user is stored in the database

User.java (Entity)

```
@Entity
@Table(name = "users")
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(unique = true, nullable = false)
    private String username;

    @Column(nullable = false)
    private String password;    // stored as BCrypt HASH, never plain text

    private String role;        // e.g. "ROLE_USER", "ROLE_ADMIN"

    // getters and setters
}
```

3.2 DTOs — what travels over HTTP

This is the class your @RequestBody maps the incoming JSON to. It has nothing to do with Spring Security's internal Authentication object — it's just a plain data holder for the raw request.

AuthRequest.java (DTO)

```
// This is the JSON body the client (Postman / React app) sends
// POST /api/auth/login
// { "username": "suraj123", "password": "MyPass@123" }

public class AuthRequest {

    private String username;
    private String password;

    // getters and setters
}
```

AuthResponse.java (DTO)

```
public class AuthResponse {

    private String token;
```

```

private String tokenType = "Bearer";

public AuthResponse(String token) {
    this.token = token;
}
// getters and setters
}

```

3.3 UserDetailsService — bridging your DB to Spring Security

Spring Security doesn't know about your User entity. You must translate it into Spring Security's own UserDetails contract. This is the single most important class you write.

MyUserDetailsService.java

```

@Service
public class MyUserDetailsService implements UserDetailsService {

    @Autowired
    private UserRepository userRepository;

    @Override
    public UserDetails loadUserByUsername(String username)
        throws UsernameNotFoundException {

        // STEP 5 in the diagram: UserDetailsService -> talks to DB
        User user = userRepository.findByUsername(username)
            .orElseThrow(() ->
                new UsernameNotFoundException("User not found: " + username));

        // Convert our own User entity into Spring Security's UserDetails
        return new org.springframework.security.core.userdetails.User(
            user.getUsername(),
            user.getPassword(), // BCrypt hash from DB
            List.of(new SimpleGrantedAuthority(user.getRole()))
        );
    }
}

```

3.4 SecurityConfig — wiring all the beans together

SecurityConfig.java

```

@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public PasswordEncoder passwordEncoder() {
        // STEP 2 in the diagram: PasswordEncoder bean
        return new BCryptPasswordEncoder();
    }

    @Bean
    public AuthenticationProvider authenticationProvider(
        MyUserDetailsService userDetailsService,
        PasswordEncoder passwordEncoder) {

```

```

// STEP 3: DaoAuthenticationProvider wires UserDetailsService + PasswordEncoder
DaoAuthenticationProvider provider = new DaoAuthenticationProvider();
provider.setUserDetailsService(userDetailsService);
provider.setPasswordEncoder(passwordEncoder);
return provider;
}

@Bean
public AuthenticationManager authenticationManager(
    AuthenticationConfiguration config) throws Exception {
    // STEP 1: exposes the AuthenticationManager (ProviderManager) as a bean
    // so our AuthController can @Autowired it
    return config.getAuthenticationManager();
}

@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http
        .csrf(csrf -> csrf.disable())
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/api/auth/**").permitAll()
            .anyRequest().authenticated()
        )
        .sessionManagement(sm -> sm
            .sessionCreationPolicy(SessionCreationPolicy.STATELESS));
    return http.build();
}
}

```

4. Step-by-Step Code Walkthrough (mapped to the diagram)

4.1 AuthController — the entry point

This is where the flow begins. Follow the **STEP A/B/C/D** comments — they map directly onto the arrows in the diagram above.

AuthController.java

```
@RestController
@RequestMapping("/api/auth")
public class AuthController {

    @Autowired
    private AuthenticationManager authenticationManager; // ProviderManager

    @Autowired
    private JwtUtil jwtUtil;

    @PostMapping("/login")
    public ResponseEntity<AuthResponse> login(@RequestBody AuthRequest request) {

        // STEP A: package raw username/password into a Spring Security "ticket"
        UsernamePasswordAuthenticationToken authToken =
            new UsernamePasswordAuthenticationToken(
                request.getUsername(),
                request.getPassword());

        // STEP B: hand the ticket to AuthenticationManager (== ProviderManager)
        // This single line triggers the ENTIRE middle section of the diagram:
        // ProviderManager -> DaoAuthenticationProvider -> UserDetailsService
        // -> PasswordEncoder.matches() -> success/failure
        Authentication authentication =
            authenticationManager.authenticate(authToken);

        // STEP C: if we reach here, authentication succeeded.
        // Store it in SecurityContextHolder (optional for stateless JWT APIs,
        // but good practice for the current request thread)
        SecurityContextHolder.getContext().setAuthentication(authentication);

        // STEP D: generate JWT token -> jjwt library
        String token = jwtUtil.generateToken(authentication.getName());

        return ResponseEntity.ok(new AuthResponse(token));
    }
}
```

The one line that matters most: `authenticationManager.authenticate(authToken)`. Everything in the middle of the diagram (`ProviderManager` → `DaoAuthenticationProvider` → `UserDetailsService` → `PasswordEncoder`) happens *inside* this single method call. If the password doesn't match, it throws `BadCredentialsException` and your controller code never reaches the next line.

4.2 JwtUtil — where the token is actually built

Once `authenticate()` returns successfully, we know the person is who they say they are. Now we hand off to `JwtUtil`, which uses the `jjwt` library to assemble the three parts of a JWT.

JwtUtil.java

```
@Component
public class JwtUtil {

    // In real projects, load this from application.properties / vault
    private final String SECRET_KEY = "my-super-secret-key-min-32-chars-long!!";
    private final long EXPIRATION = 1000 * 60 * 60; // 1 hour

    private Key getSigningKey() {
        return Keys.hmacShaKeyFor(SECRET_KEY.getBytes(StandardCharsets.UTF_8));
    }

    // STEP E: called by AuthController after successful authentication
    public String generateToken(String username) {
        return Jwts.builder()
            .setSubject(username) // Payload: "sub"
            .claim("role", "ROLE_USER") // custom claim
            .setIssuedAt(new Date()) // Payload: "iat"
            .setExpiration(new Date(System.currentTimeMillis() + EXPIRATION))
            .signWith(getSigningKey(), SignatureAlgorithm.HS256) // Signature
            .compact(); // jjwt builds
        //
        Header.Payload.Signature
    }

    public String extractUsername(String token) {
        return getClaims(token).getSubject();
    }

    public boolean isValid(String token, UserDetails userDetails) {
        String username = extractUsername(token);
        return username.equals(userDetails.getUsername()) && !isExpired(token);
    }

    private boolean isExpired(String token) {
        return getClaims(token).getExpiration().before(new Date());
    }

    private Claims getClaims(String token) {
        return Jwts.parserBuilder()
            .setSigningKey(getSigningKey())
            .build()
            .parseClaimsJws(token)
            .getBody();
    }
}
```

4.3 JwtAuthFilter — how future requests get recognised

Login only happens once. Every request *after* that carries the token in the `Authorization: Bearer <token>` header. A custom filter reads it, and re-populates the `SecurityContextHolder` for that request — without hitting the database or checking a password again.

JwtAuthFilter.java

```
@Component
public class JwtAuthFilter extends OncePerRequestFilter {

    @Autowired private JwtUtil jwtUtil;
    @Autowired private MyUserDetailsService userDetailsService;

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain filterChain)
        throws ServletException, IOException {

        String header = request.getHeader("Authorization");

        if (header != null && header.startsWith("Bearer ")) {
            String token = header.substring(7);
            String username = jwtUtil.extractUsername(token);

            if (username != null &&
                SecurityContextHolder.getContext().getAuthentication() == null) {

                UserDetails userDetails =
                    userDetailsService.loadUserByUsername(username);

                if (jwtUtil.isTokenValid(token, userDetails)) {
                    UsernamePasswordAuthenticationToken authToken =
                        new UsernamePasswordAuthenticationToken(
                            userDetails, null, userDetails.getAuthorities());
                    SecurityContextHolder.getContext().setAuthentication(authToken);
                }
            }
        }
        filterChain.doFilter(request, response);
    }
}
```

5. Full Request/Response Trace

HTTP Request & Response

```
// Request -> POST /api/auth/login
{
  "username": "suraj123",
  "password": "MyPass@123"
}

// Response <- 200 OK
{
  "token":
  "eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJzdXJhbjEyMyIsInJvbmGU0iOiJST0xFOX1VTRVIiLCJpYXQiOiJE3NTIxMjM0NTYsImV4cCI6MTc1MjEyNzA1Nn0.4f8s...",
  "tokenType": "Bearer"
}
```

#	Who	Action
1	Client	Sends { "username": "suraj123", "password": "MyPass@123" } to / api/auth/login
2	AuthController	Wraps it into UsernamePasswordAuthenticationToken (currently "unauthenticated")
3	AuthenticationManager (ProviderManager)	Loops through registered providers, finds one that supports () this token type
4	DaoAuthenticationProvider	Calls UserDetailsService.loadUserByUsername("suraj123")
5	MyUserDetailsService	Queries DB, returns UserDetails with the stored BCrypt hash
6	DaoAuthenticationProvider	Calls passwordEncoder.matches("MyPass@123", storedHash)
7	ProviderManager	Returns a fully-populated, authenticated Authentication object
8	AuthController	Stores it in SecurityContextHolder, then calls jwtUtil.generateToken()
9	JwtUtil + jjwt	Builds Header + Payload, signs with HMAC-SHA256, returns the compact token string
10	Client	Receives { "token": "...", "tokenType": "Bearer" }, stores it, sends it on every future call

6. JWT Structure Refresher

A JWT is just three Base64URL-encoded parts joined by dots: `Header.Payload.Signature`

Part	Contains	In our code
Header	Algorithm + token type — <code>{"alg": "HS256", "typ": "JWT"}</code>	Set automatically by <code>signWith(key, HS256)</code>
Payload	Claims: <code>sub</code> (username), <code>role</code> , <code>iat</code> , <code>exp</code>	<code>.setSubject(), .claim(), .setIssuedAt(), .setExpire</code>
Signature	HMAC of Header+Payload using the secret key	<code>.signWith(getSigningKey(), HS256)</code>

Important: the payload is only *encoded*, not encrypted. Anyone can decode and read it (try pasting a token into jwt.io) — never put passwords or sensitive data in the payload. The signature only proves it wasn't *tampered with*, it doesn't hide the content.

7. Interview Q&A

Q1. Why is `AuthenticationManager` an interface and `ProviderManager` the implementation?

So Spring can support multiple authentication mechanisms (DB, LDAP, OAuth2) behind one contract. Your controller only ever talks to the interface; `ProviderManager` decides internally which `AuthProvider` actually knows how to verify that specific type of credential.

Q2. Why can't we just compare passwords with `.equals()`?

Passwords are never stored in plain text. `BCryptPasswordEncoder` stores a salted hash, and a hash of the same password is different every time it's generated. `matches(raw, hash)` re-hashes the raw input with the same salt (extracted from the stored hash) and compares the results — that's the only correct way.

Q3. Where is the `SecurityContextHolder` used in a stateless JWT API if we don't use sessions?

It's still populated per-request, just not persisted across requests. On login, it holds the Authentication result briefly. On every subsequent request, `JwtAuthFilter` repopulates it fresh from the JWT so that `@PreAuthorize` and `SecurityContextHolder.getContext().getAuthentication()` work correctly inside that one request's lifecycle.

Q4. What happens if authentication fails?

`authenticationManager.authenticate()` throws an exception (e.g. `BadCredentialsException` or `UsernameNotFoundException`). This propagates up and, if unhandled, Spring's `ExceptionHandlerFilter` converts it into a 401/403 response. In a REST API you typically catch it in a `@ControllerAdvice` and return a clean JSON error.

Q5. Why HS256 vs RS256 for signing?

HS256 is symmetric — one secret key both signs and verifies, simple for a single-service app. RS256 is asymmetric — a private key signs, a public key verifies, so other services can validate the token without ever holding the secret that can create one. Microservice architectures typically prefer RS256.

Q6. Can a JWT be revoked before it expires?

Not natively — that's JWT's biggest tradeoff versus sessions. Common workarounds: keep a short expiry with a refresh-token rotation flow, or maintain a server-side denylist/Redis blacklist of revoked token IDs checked in the filter.

8. Cheat Sheet

- **Login entry point:** `authenticationManager.authenticate(new UsernamePasswordAuthenticationToken(user, pass))`
- **Who fetches the user:** `UserDetailsService.loadUserByUsername()`
- **Who checks the password:** `PasswordEncoder.matches(raw, hash)` inside `DaoAuthenticationProvider`
- **Who builds the token:** your `JwtUtil`, using `Jwts.builder()...signWith(key, HS256).compact()`
- **Who reads the token on future requests:** a custom `OncePerRequestFilter` (e.g. `JwtAuthFilter`)
- **JWT = Header.Payload.Signature** — encoded, not encrypted
- **Stateless config:** `SessionCreationPolicy.STATELESS` in `SecurityFilterChain`